Kevin Küster

Zusammenfassung: Syntax

Syntax

Compiler

Funktion: Umwandlung Quellprogramm [Hochsprache] ⟶ Zielprogramm [Maschinensprache]

▷ Generierung von Meldungen (Hinweise, Fehler, ...)

Analyse:

(i) Lexikalische Analyse - Scanner ↦ Symbolfolge

(ii) Syntaktische Analyse - Parser ↦ Syntaxbaum

(iii) Semantische Analyse ↦ dekorierter Ableitungsbaum

Synthese:

▷ Zwischencodeerzeugung

▷ Codeerzeugung

↳ Zielprogramm

LL(k) - Analyse

- Spezialisierung d. Top-Down-Analyse
  (LL: „Scanning form left, using left derivations")
- Wenn für Nichtterminalsymbol mehrere Expansionen möglich:
  entscheide für nächste Expansion aufgrund der nächsten
  k Eingabesymbole (Lookahead)
  ↳ Nur für LL(k)-Grammatiken möglich
- Praktikable Charakterisierung: SLL(k) („stark LL(k)")

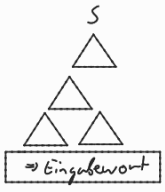
(Syntax-) Analysephase

Funktion: Quelltextprogramm → Strukturbaum

Falls Programmiersprache als formale Sprache
↳ Ableitung für Programm via Produktionen

Top-Down-Syntaxanalyse

Vom Startsymbol ausgehend wird versucht,
Eingabewort abzuleiten



Entscheidungsproblem:
Welche Prod. ist bei
alternativen anzuwenden

k=1 - Problem

- Gemeinsamer Präfix
  ↳ Zwei alternativen können mit derselben Symbolfolge beginnen:

$N \to \alpha\beta \mid \alpha\gamma \quad \Rightarrow$ Lösung: Linksfaktorisierung $N \to \alpha M \,,\; M \to \beta \mid \gamma$

Beispiel

<Stmt> → if <Expr> then <Stmt> fi |
if <Expr> then <Stmt> else <Stmt> fi

Nach Transformation:

<Stmt> → if <Expr> then <Stmt> IfEnd

IfEnd → fi | else Stmt fi

SLL(k) - Bedingungen

Grammatik SSL(k), falls für alle Prod. der Form $A \to \gamma \mid \gamma'$:

$first_k(\gamma\, follow_k(A)) \cap first_k(\gamma'\, follow_k(A)) = \emptyset$

↳ Wenn γ und γ' nur terminale Zeichenreihen mit mind. k Zeichen ableitbar; $follow_k(A)$ enthällt (keine Auswirkung auf Erg.)
⇒ Für k=1 und G ε-frei

▷ Jede SLL(k)-Grammatik ist LL(k)-Grammatik

▷ Jede LL(1)-Grammatik ist SLL(1)-Grammatik

LL(k) - Grammatik

Kontextfreie Grammatik $G = (N, T, P, S)$ LL(k)-Grammatik, falls:

Für $k \geq 1$, wenn für alle Paare von Ableitungen:

$$S \Rightarrow_l^* \alpha A \beta \Rightarrow \alpha \gamma \beta \Rightarrow_l^* \alpha \delta$$

$$S \Rightarrow_l^* \alpha A \beta \Rightarrow \alpha \gamma' \beta \Rightarrow_l^* \alpha \delta'$$

dann: $(k:\delta = k:\delta') \Rightarrow \gamma = \gamma'$

↳ $k:x \wedge x \in N \wedge x \in T^*$ ist k-Anfang von x:

(i) $k:x = x$, wenn $|x| < k$

(ii) $k:x = u$, wenn $x = uv \wedge |u| = k$

⇒ Funktion: Unter Historie α kann aus den nächsten k Zeichen die nächste anzuwendene Produktion eindeutig bestimmen.

SLL(k) - Grammatik

Kontextfreie Grammatik $G = (N, T, P, S)$ SLL(k) - Grammatik, falls:

Für $k \geq 1$, wenn für alle Paare von Ableitungen:

$$S \Rightarrow_l^* \alpha A \beta \Rightarrow \alpha \gamma \beta \Rightarrow_l^* \alpha \delta$$

$$S \Rightarrow_l^* \alpha' A \beta' \Rightarrow \alpha' \gamma' \beta' \Rightarrow_l^* \alpha' \delta'$$

dann: $(k:\delta = k:\delta') \Rightarrow \gamma = \gamma'$

↳ $k:x \wedge x \in N \wedge x \in T^*$ ist k-Anfang von x:

(i) $k:x = x$, wenn $|x| < k$

(ii) $k:x = u$, wenn $x = uv \wedge |u| = k$

⇒ Funktion: Ohne Historie α kann aus den nächsten k Zeichen die nächste anzuwendene Produktion eindeutig bestimmen.

First & Follow Mengen

Grammatik $G' = (N', T', P', S')$, Startzeichen $S' \notin N$; Endzeichen $\$ \in T$;

$N' = N \cup \{S\}$, $T' = T \cup \{\$\}$, $P' = P \cup \{S' \to S\$\}$, dann:

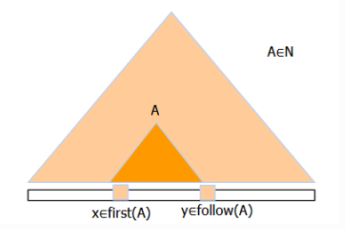
$first_k(\alpha) = \{k:w \mid \alpha \Rightarrow^* w \in T'^*\}$, $\alpha \in V'^*$

↳ Menge aller k-Anfänge von Terminalsequenzen, die aus α ableitbar sind

$follow_k(A) = \{w \mid \exists \alpha, \beta \in V'^* : S' \Rightarrow^* \alpha A \beta, w \in first_k(\beta)\}$, $A \in N$

↳ Menge aller k-Anfänge von Terminalsequenzen, die in einer Ableitung von S' unmittelbar auf A folgen können

▷ Notation: Ohne Index ⇒ $k = 1$



first(A) gibt an, mit welchen Terminalen die aus A ableitbaren Satzformen beginnen können, bzw. ob A nach ε ableitbar ist

Follow(A) gibt an, welche Terminalsymbole in beliebigen (aus S ableitbar) Satzformen unmittelbar nach A folgen können (bzw. $)

Berechnung - First

▷ first(X) $\forall X \in V'$ durch Erweiterung:

(i) setze $first(a) = \{a\}$ $\forall a \in T'$

(ii) setze $first(A) = \emptyset$ $\forall A \in N'$

(iii) $\forall p \in P'$: $A \to \varepsilon$ füge ε zu first(A) hinzu

(iv) Wiederhole, bis keine neue Änderung:

$\forall p \in P'$: $A \to X_1 X_2 \dots X_n$, $\forall i \in \{1, \dots, n\}$: $X_i \in V'$:

▷ $\exists k : a \in first(X_k) \wedge T' \wedge \forall i < k : \varepsilon \in first(X_i)$:

füge a zu first(A) hinzu

▷ $\forall i \in \{1, \dots, n\} : \varepsilon \in first(X_i)$:

füge ε zu first(A) hinzu

⇒ Rekursive Berechnung $first(\alpha)$, $\alpha \in V'^*$:

(i) $first(\varepsilon) = \{\varepsilon\}$

(ii) $\alpha = X\beta$, $X \in V'$:

$\varepsilon \notin first(x)$: $first(\alpha) = first(x)$

$\varepsilon \in first(x)$: $first(\alpha) = (first(x) \setminus \{\varepsilon\}) \cup first(\beta)$

Berechnung - Follow

▷ follow(A) $\forall A \in N'$ durch Erweiterung:

(i) setze $follow(A) = \emptyset$ $\forall A \in N'$

(ii) setze $follow(S) = \{\$\}$

(iii) Wiederhole, bis keine neue Änderung:

$\forall p \in P'$: $A \to \alpha B \beta$, $B \in N \wedge \beta \in V'^*$:

▷ Füge alle $first(\beta) / \{\varepsilon\}$ zu follow B hinzu

▷ $\varepsilon \in first(\beta)$: Füge alle follow(A) zu follow(B) hinzu

Beispiel

$E \to E + T \mid T$

$T \to T * F \mid F$

$F \to (E) \mid id$

Linksrekursion auflösen ⟶

$E \to TE'$

$E' \to +TE' \mid \varepsilon$

$T \to FT'$

$T' \to *FT' \mid \varepsilon$

$F \to (E) \mid id$

$S \to E\$$

$first(E) = first(T) = first(F) = \{id, (\}$

$first(E') = \{+, \varepsilon\}$

$first(+TE') = \{+\}$

$first(T') = \{*, \varepsilon\}$

$follow(E) \to \{\$,)\}$

$follow(E') = \{\$,)\}$

$follow(T) = \{\$,), +\}$

$follow(T') = \{\$,), +\}$

$follow(F) = \{\$,), +, *\}$

LL-Parser

Top-Down-Parser, der Eingabe von links nach rechts abarbeitet, um die Linksableitung der Eingabe zu berechnen

---

LL(1)-Parser - Rekursiver Abstieg

▷ Definiere $\forall X \in N$ Prozedur X

↳ Für alternative: $X \to X_1 | X_2 | \dots | X_n$:

switch s { case first(X_1, follow(X)): Code X_1;
...
case first(X_n, follow(X)): Code X_n;
default: error(...); }

Für rechte Seite $X_i = Y_1, \dots, Y_m$ erzeuge: $C_i; \dots; C_m$; return;

↳ $Y_i \in T$: C_i = if (s == Y_i) s = nextSymbol() else error(...);

↳ $Y_i \in N$: C_i = Y_i ();

LL-Parser - Kellerautomat

Initialisierung mit Satzform: S $

↳ Streichen von $t \in T$ aus Keller, wenn diese mit jeweiligem Anfang des Resteingabeworts übereinstimmen

↳ Expandieren von $n \in N$ aus Keller in Abhängigkeit von ersten Zeichen des Resteingabewortes

↳ Analyse erfolgreich, falls Keller: [$] mit Resteingabe: $

Steuertabelle M: $N \times T' \to$ Aktion

Steuertabelle LL(1)-Grammatik

Steuertabelle $M: N \times T' \to P$

Algorithmus:

(i) $\forall p \in P: A \to \alpha$:

↳ $\forall a \in$ first(α) $\wedge\, T$: trage $A \to \alpha$ in M[A,a] ein

Falls $\varepsilon \in$ first(α): $\forall b \in$ follow(A): trage $A \to \alpha$ in M[A,b] ein

(ii) Freie Fleder ≙ error-Eintrag

⇒ G LL(1)-Grammatik, falls Steuermatrix keine Mehrfacheinträge enthält

Eingabesymbol

Nichtterminale	id	+	*	(	)	$
E	E→TE'			E→TE'		
E'		E'→+TE'			E'→ε	E'→ε
T	T→FT'			T→FT'		
T'		T'→ε	T'→*FT'		T'→ε	T'→ε
F	F→id			F→(E)		

Beispiel

$E \to TE'$

$E' \to +TE' \mid \varepsilon$

$T \to FT'$

$T' \to *FT' \mid \varepsilon$

$F \to (E) \mid id$

$E \to TE'$	first(E) = first(T)	follow(E) → {$,)}
$E' \to +TE' \mid \varepsilon$	= first(F) = {id, (}	follow(E') = {$,)}
$T \to FT'$	first(E') = {+, ε}	follow(T) = {$,), +}
$T' \to *FT' \mid \varepsilon$	first(+TE') = {+}	follow(T') = {$,), +}
$F \to (E) \mid id$	first(T') = {*, ε}	follow(F) = {$,), +, *}

⇒ $E \to TE'$

$E' \to +TE' \mid \varepsilon$ — first = {+}; follow(E') = {$,)}

$T \to FT'$

$T' \to *FT' \mid \varepsilon$ — first = {*}; follow(T') = {$,), +}

$F \to (E) \mid id$ — first = {(}; first = {id}

LL(1)-Parser - Kellerautomat

- Initialisierung: Eingabewort: $
- Ausgabeband: ε
- Keller: push $, push S

Sei $s \in T'$ erstes Zeichen Resteingabewort
Sei $q \in N \cup T'$ oberstes Kellersymbol
} Aktion in Abhängigkeit

Wenn $q \in N$:

↳ Wenn M[q,s] = $q \to X_1, \dots, X_n$:

pop; push(X_n); ...; Push(X_1); Output($q \to X_1, \dots, X_n$)

sonst: error

Wenn $q \in T$, $q \neq$ $

↳ Wenn q = s :

pop; s = readNext()

Sonst: error

Wenn q = $

↳ wenn s = $: accept

sonst: error

Beispiel

$E \to TE'$

$E' \to +TE' \mid \varepsilon$

$T \to FT'$

$T' \to *FT' \mid \varepsilon$

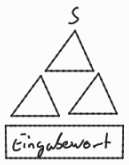
$F \to (E) \mid id$

Analyse: id+id

	Satzform	Restwort	Produktion
Expansionsschritt	E	id + id$	E→T E'
	T E'	id + id$	T→F T'
	F T' E'	id + id$	F→id
Leseschritt	id T' E'	id + id$	
	T' E'	+ id$	T'→ε
	E'	+ id$	E'→+ T E'
	+ T E'	+ id$	
	T E'	Id$	T→F T'
	F T' E'	Id$	F→id
	id T' E'	Id$	
	T' E'		T'→ε
	E'		E'→ε

Bottom-Up-Syntaxanalyse

Von Eingabewort aus zum Startsymbol reduzieren



↳ Shift-Reduce

Eingabetoken lesen:

- shift auf Stack
- Reduktion Elemente auf Stack und ersetzen durch linke Seite der Produktion

↳ LR-Syntaxanalyse:

- L: Eingabewort von links → rechts untersuchen
- R: Konstruktion Rechtsableitung

Beispiel

Beispiel: Analyse von id+id*id

Schritt	Kellerinhalt	Eingabewort	Aktion	
0		id + id * id $	shift	id
1	id	+ id * id $	reduce	F→id
2	F	+ id * id $	reduce	T→F
3	T	+ id * id $	reduce	E→T
4	E	+ id * id $	shift	+
5	E +	id * id $	shift	id
6	E + id	* id $	reduce	F→id
7	E + F	* id $	reduce	T→F
8	E + T	* id $	shift	*
9	E + T *	id $	shift	id
10	E + T * id	$	reduce	F→id
11	E + T * F	$	reduce	T→T * F
12	E + T	$	reduce	E→E + T
13	E	$	shift	$
14	E $		reduce	S→E $

Grammatik

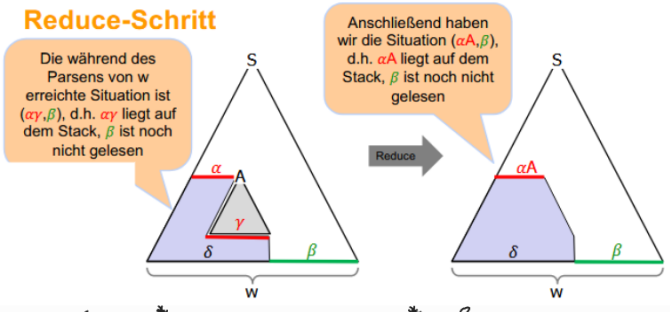
S → E$
E → E + T
E → T
T → T * F
T → F
F → (E)
F → id

Entscheidung ob shift oder reduce bei T

Reduce-Schritt

Die während des Parsens von w erreichte Situation ist ($\alpha\gamma$,β), d.h. $\alpha\gamma$ liegt auf dem Stack, β ist noch nicht gelesen

Anschließend haben wir die Situation (αA,β), d.h. αA liegt auf dem Stack, β ist noch nicht gelesen



$\Delta\ S \underset{R}{\Rightarrow}^* \alpha A \beta \underset{R}{\rightarrow} \alpha\gamma\beta \underset{R}{\Rightarrow}^* \delta\beta = w$,

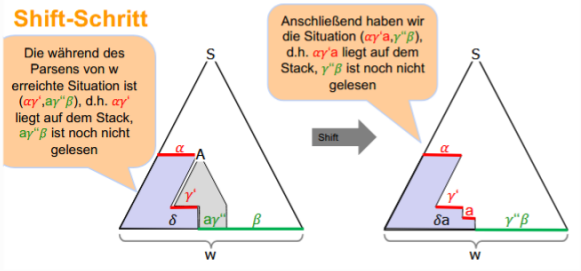
$\alpha, \gamma \in V^*, w, \beta, \delta \in T^*, (A \rightarrow \gamma) \in P$

$\Rightarrow$ Reduktionsklasse $R_{A\rightarrow\gamma}$

Shift-Schritt

Die während des Parsens von w erreichte Situation ist ($\alpha\gamma'$,$a\gamma''\beta$), d.h. $\alpha\gamma'$ liegt auf dem Stack, $a\gamma''\beta$ ist noch nicht gelesen

Anschließend haben wir die Situation ($\alpha\gamma'a$,$\gamma''\beta$), d.h. $\alpha\gamma'a$ liegt auf dem Stack, $\gamma''\beta$ ist noch nicht gelesen



$\Delta\ S \underset{R}{\Rightarrow}^* \alpha A \beta \underset{R}{\Rightarrow}^* \alpha\gamma' a \gamma'' \beta \underset{R}{\Rightarrow} \delta a \gamma \beta = w$, $a, \gamma' \in V^*$, $A \in N$, $a \in T$, $w, \beta, \delta, \gamma'' \in T^*$, $A \underset{R}{\Rightarrow}^* \gamma' a \gamma''$ $\not\Rightarrow\ \gamma' B a \gamma'' \Rightarrow \gamma' a \gamma''$

$\Rightarrow$ Shiftklasse R_{shift}

Satz: Die Reduktionsklassen einer kontextfreien Grammatik G sind paarweise disjunkt gdw. G eindeutig ist

---

k-Kellerklassen

Lookahead auf $k \in \mathbb{N}$ → Kellerklasse R^k_r:

$R^k_r = \{(x,z) \mid \exists (x,y) \in R_r \wedge z = k{:}y\}$, $r \in P \cup \{shift\}$

↳ LR(k)-Parser betrachtet ersten k Zeichen

⇒ Situation (x, z)

- x: aktueller Kellerinhalt
- z: nächsten k Eingabezeichen

shift, falls $(x,z) \in R^k_{shift}$

reduce, falls $(x,z) \in R^k_p$

⇒ Grammatik **LR(k)-Grammatik**, wenn ihre k-Kellerklassen paarweise disjunkt sind

LR(k)-Steuerautomat

Deterministischer endlicher Automat mit Zustandsmenge Q und Funktion action: $Q \times T'^k \rightarrow 2^{P \cup \{shift\}}$

↳ Parser in Situation (x, z)

↳ Automat erreicht mit Wort x Zustand q

↳ $(x,z) \in R^k_r$ gdw. $r \in$ action (q,z)

Aufteilung in zwei Tabellen: (- oberstes Kellerelement = aktueller Zustand; - Aktionstabelle: Verarbeitung Zustand × Eingabe)

(i) goto: $Q \times V \rightarrow Q$

(ii) action: $Q \times T'^k \rightarrow P \cup \{shift\}$

Aktionen: (i) Shift(z): Eingabesymbol auf Keller, nächster Zustand: z

(ii) Reduce(n): Anwendung Regel n, nächster Zustand: via goto-Tabelle

(iii) Accept: Eingabewort akzeptiert

(iv) Error: Nicht beschriebene Eingabekombination → Wort abgelehnt

---

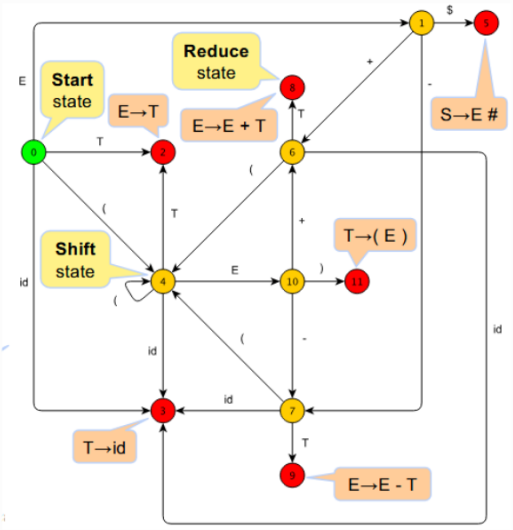
(S) LR(1)-Algorithmus

```
s=Anfangszustand; ERFOLG=false; FEHLER=false;
lege s auf den Stapel
repeat
  sei t nächstes Symbol;
  s Element an der Spitze des Stapels;
  case ACTION(s,t) of
    accept: ERFOLG = true;
    error: FEHLER = true;
    shift: lege t auf Stapel;
           lege GOTO(s,t) auf Stapel;
           beschaffe nächstes lexikalisches Element;
    reduce A→b: entferne 2*|b| Elemente vom Stapel;
           s' sei jetzt die Spitze des Stapels;
           lege A auf den Stapel;
           lege GOTO(s',A) auf Stapel;
  end case
until ERFOLG or FEHLER
```

Grammatik

S → E$
E → E + T
E → E - T
E → T
T → (E)
T → id

Q	goto id	+	-	(	)	$	E	T	action
0	3			4			1	2	shift
1		6	7			5			shift
2									E→T
3									T→id
4	3			4			10	2	shift
5									S→E$
6	3			4				8	shift
7	3			4				9	shift
8									E→E+T
9									E→E-T
10		6	7		11				shift
11									T→(E)



Konstruktion LR(0) Steuerautomat

(i) Eingabewort muss aus S ableitbar sein

(ii) Anfangs kein Zeichen eingelesen; Restwort aus $E\$$ ableitbar
↳ $[S \to \bullet E\$]$; "•" trennt bearbeitete Teile
↳ $E \to T$ oder $E \to E+T \Rightarrow [E \to \bullet E+T]$ & $[E \to \bullet T]$

(iii) Startzustand ist Abschluss von $[S \to E\$]$

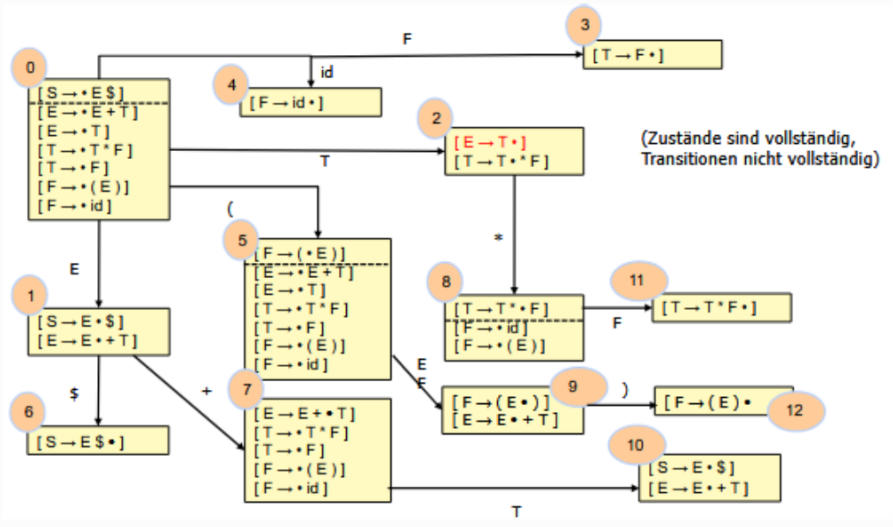
- Restwort muss aus einem auf den "•" folgenden Worten ableitbar sein
- Shifts von Terminalzeich erster Zustand: "(", "id"
- Bei Reduktion können von Startzustand "E", "T", "F" auf Keller (→ Folgezustände)
- Über Verschiebung "•" entstehen Folgezustände ⇒ Shift-Schritte
- Wenn "•" ganz rechts: Gesamte Rechte Seite auf Stack ⇒ Reduce folgt
- Endzustände: Eine Aktion → sonst keine Terminierung möglich

Grammatik

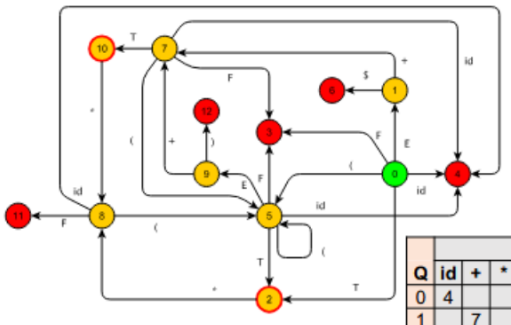
S → E$
E → E + T
E → T
T → T * F
T → F
F → (E)
F → id

Startzustand:

[S → • E $]
[E → • E + T]
[E → • T]
[T → • T * F]
[T → • F]
[F → • (E)]
[F → • id]



(Zustände sind vollständig, Transitionen nicht vollständig)



- Aber: follow(E) = { +,), $ }
  follow(T) = { *, +,), $ }
  follow(F) = { *, +,), $ }
- Damit kann eine Reduktion von A→γ nur erfolgen, wenn das nächste Eingabezeichen in follow(A) liegt

Grammatik

S → E$
E → E + T
E → T
T → T * F
T → F
F → (E)
F → id

	goto								action
Q	id	+	*	(	)	$	E	T	F	
0	4			5			1	2	3	shift
1		7				6				shift
2			8							shift, E→T
3										T→F
4										F→id
5	4			5			9	2	3	shift
6										S→E$
7	4			5				10	3	shift
8	4			5					11	E→E+T
9		7			12					E→E-T
10			8							shift, E→E+T
11										T→T*F
12										F→(E)